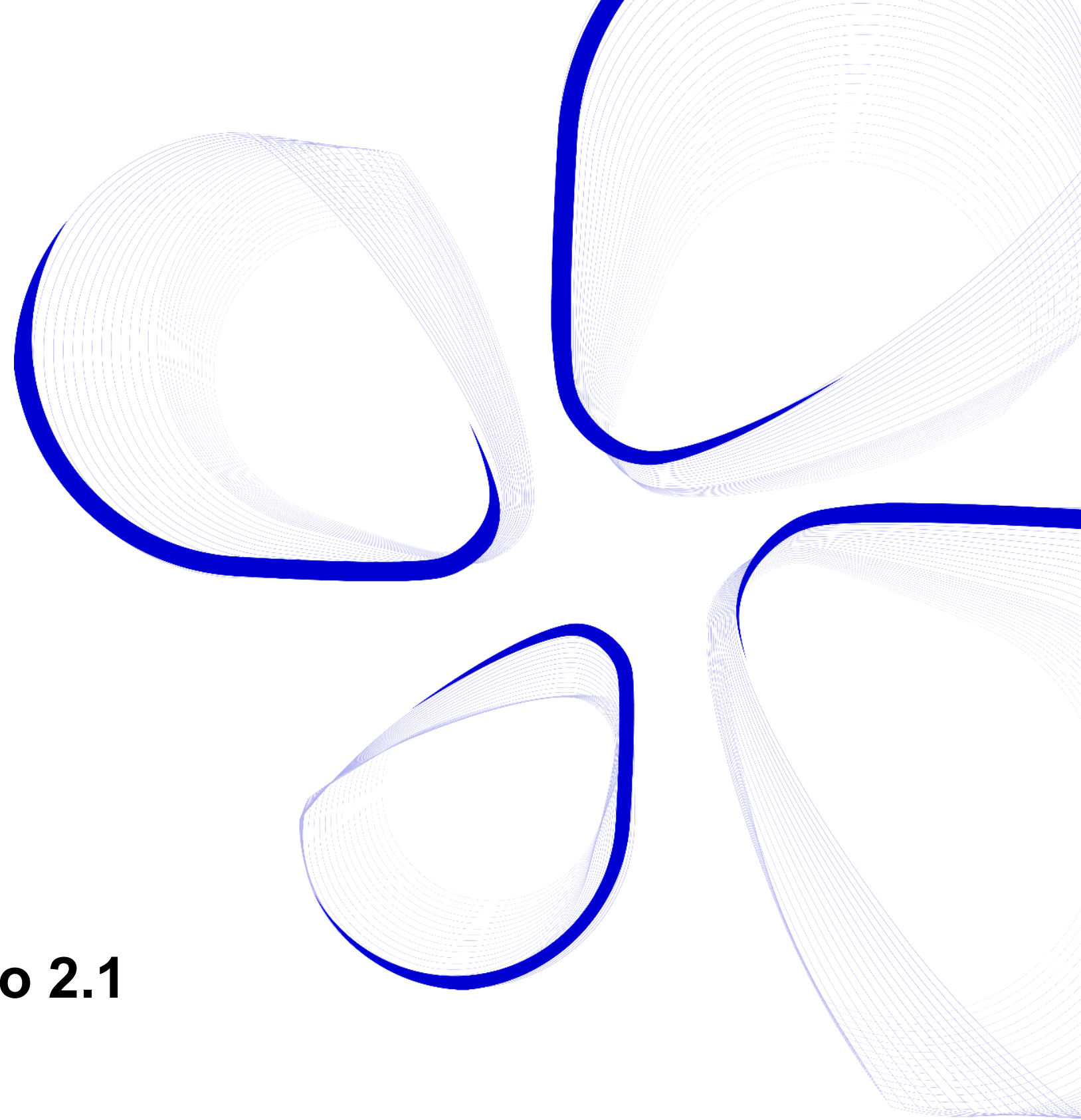




AyesaFlix

Documentación Proyecto Grupo 2.1

Fecha de última actualización: 02/06/2026



Control de documentación

Hoja de control

Realizado	Revisado	Aprobado
Miguel Polvillo Acevedo		
Carlos León Saavedra		
Daniel Vázquez Chinchilla		
Alejandro Aguilar Santaella		
Víctor Coquelet Salguero		
Iván Zamora Jiménez		
Abderrazak Oumouss Ait Moussa		
Miguel Castellero Gómez		

Control de modificaciones

Versión	Fecha	Control de Modificaciones	Autor
1.0	13/05/2026	Realización de la primera documentación básica del proyecto	Miguel Polvillo Acevedo Carlos León Saavedra Daniel Vázquez Chinchilla Alejandro Aguilar Santaella Víctor Coquelet Salguero Iván Zamora Jiménez Abderrazak Oumouss Ait Moussa Miguel Castellero Gómez
2.0	02/06/2026	Nueva información y reestructuración del documento	Miguel Polvillo Acevedo, Iván Zamora Jiménez
2.1	02/06/2026	Corrección de errores	Miguel Polvillo Acevedo

Contenido

[1] **Introducción1**

 [1.1] Propuesta de Valor1

 [1.2] Público Objetivo (Target)1

[2] **Stack utilizado.....2**

 [2.1] Back-End.....2

 [2.2] Base de Datos2

 [2.3] Front-End2

 [2.4] Herramientas.....2

[3] **Funcionalidades del MVP3**

 [3.1] Exploración y Consulta del Catálogo3

 [3.2] Gestión y mantenimiento de Datos (CRUD)3

 [3.3] Watchlist (lista de pendientes)3

 [3.4] Favoritas.....3

 [3.5] Reseña personal3

 [3.6] Estadísticas3

 [3.7] Paginación.....3

[4] **API REST y Modelo de Datos4**

 [4.1] Modelo de datos4

 [4.2] Endpoints de Películas (/api/peliculas)5

 [4.3] Endpoints de Géneros (/api/generos)5

 [4.4] Endpoint de Estadísticas (/api/estadisticas).....5

[5] **Diagramas UML.....6**

 [5.1] Diagrama de Caso de Uso.....6

 [5.2] Diagrama de Clases6

 [5.3] Diagrama de Arquitectura7

 [5.4] Diagrama de Secuencia: Carga de la Cartelera.....7

 [5.5] Diagrama de Secuencia: Modificar Valoración (Fetch)8

 [5.6] Diagramas de Flujo9

[6] **Guía despliegue19**

[7] **Equipo20**

 [7.1] Backend20

 [7.2] Base de Datos20

 [7.3] Front-End20

[1] Introducción

La aplicación tiene como objetivo principal ofrecer una plataforma centralizada para la gestión, consulta y organización de un catálogo cinematográfico. A través de una interfaz web intuitiva, el sistema permite registrar y administrar información detallada sobre películas, facilitando tanto la búsqueda como la exploración del contenido disponible.

El proyecto sigue una arquitectura por capas (Controlador – Servicio – Repositorio – Entidad) construida con Spring Boot, que expone una API REST que intercambia datos en formato JSON. El frontend es una capa estática e independiente (HTML5, Bootstrap 5 y JavaScript con la Fetch API) que consume dicha API, y la persistencia se realiza sobre una base de datos H2 en memoria. Esta combinación tecnológica garantiza una solución sencilla, desacoplada y alineada con los estándares actuales del desarrollo de software.

A lo largo de este documento se detallan los requisitos funcionales y no funcionales del sistema, el modelo de datos, la arquitectura de la aplicación, las funcionalidades implementadas y las decisiones técnicas adoptadas durante el proceso de desarrollo.

[1.1] Propuesta de Valor

Nuestra aplicación es un gestor local de películas diseñado para centralizar, organizar y puntuar tu historial cinematográfico sin depender de internet ni de plataformas externas. La gestión del catálogo funciona localmente sin depender de servicios externos ni de conexión a Internet, aunque requiere que la aplicación esté ejecutándose en el equipo local. Los tráileres sí requieren acceso a Internet. A diferencia de las redes sociales de cine tradicionales, ofrecemos un espacio privado y sin ruido donde el único criterio que importa es el tuyo. Te permite saber al instante qué películas te gustaron, cuáles no, y mantener un registro fiel de tus preferencias para decidir siempre qué ver a continuación, con total privacidad y control de tus datos.

Beneficios clave que aporta:

- **Privacidad total:** Al funcionar de forma local, los datos de los usuarios no se comparten, no se venden, ni dependen de un servidor en la nube.
- **Criterio puro:** Sin sesgos de taquilla, críticas profesionales o algoritmos comerciales. El sistema se adapta estrictamente al gusto del usuario.
- **Simplicidad y control:** Una interfaz limpia para registrar, puntuar y consultar de un vistazo si una película vale la pena volver a verla o no.

[1.2] Público Objetivo (Target)

El público objetivo no es el "espectador casual" que ve una película al año, sino un perfil más específico que valora el orden y su propio criterio. Lo podemos dividir en tres perfiles principales:

El Cinéfilo Organizado (El "Coleccionista")

- **Perfil:** Personas que ven muchas películas a la semana y les gusta llevar un control estricto de lo que han visto para no repetir por error o para recordar títulos específicos.
- **Necesidad:** Un "diario de a bordo" digital, rápido de usar y que no les obligue a crear cuentas en páginas web.

Grupos Familiares o de Amigos (Uso Local Compartido)

- **Perfil:** Hogares donde varias personas comparten la misma pantalla u ordenador y quieren saber el veredicto de los demás miembros de la casa antes de poner una película.
- **Necesidad:** Un registro local donde consultar rápidamente

Usuarios Celosos de su Privacidad

- **Perfil:** Personas que huyen de las redes sociales tradicionales (como Letterboxd o IMDb), que no quieren que sus datos de consumo estén en la nube y que prefieren aplicaciones offline que funcionen rápido y consuman pocos recursos.
- **Necesidad:** Herramientas locales, eficientes, que proporcionan soberanía total sobre los datos del usuario.

[2] Stack utilizado

[2.1] Back-End

Java 8 (1.8)

Spring Boot 2.7.18 (Spring Web / MVC)

Spring Data JPA (Hibernate)

Spring Boot Validation (Bean Validation)

Springdoc-openapi (Swagger UI) 1.7.0

JUnit 5 / Spring Boot Test (Framework para la ejecución de pruebas unitarias de la API)

[2.2] Base de Datos

Por requisitos del proyecto, se utiliza H2 en memoria como sistema de persistencia temporal. En un entorno productivo podría sustituirse por una base de datos persistente (MySQL, PostgreSQL, etc.) sin modificar la arquitectura de la aplicación.

[2.3] Front-End

HTML5

CSS3

Bootstrap 5

JavaScript (Fetch API).

Frontend estático (sin framework), desacoplado del backend, que consume la API REST mediante llamadas fetch.

[2.4] Herramientas

Spring Tool Suite 4 (IDE)

Git/ GitLab (Control de versiones y alojamiento del repositorio)

Apache Maven / Maven Wrapper (Gestor de dependencias y construcción)

Trello

Draw.io

Swagger UI (springdoc-openapi) - documentación interactiva de la API

Postman (colección incluida en /documentacion)

[3] Funcionalidades del MVP

[3.1] Exploración y Consulta del Catálogo

Página principal en la que saldrán las películas en diferentes tarjetas, en la cuales verás la información básica de esta con la posibilidad de valorar cada película con una puntuación de 1 a 5 estrellas. Si haces clic en la tarjeta, se ampliará y saldrá toda la información de la película (título, duración, fecha de estreno, reparto, sinopsis, género, tráiler, valoración y reseña personal).

Esta página principal también contará con una barra de búsqueda, la cual te permitirá buscar diferentes películas en base a su nombre (título).

DETALLE DE PELÍCULA

Al hacer clic en una tarjeta, el usuario accederá a la ventana emergente, donde se mostrará la información completa: título, sinopsis, duración, fecha de estreno, género y reparto principal.

BUSCADOR Y FILTROS

La página principal contará con una barra de búsqueda que permitirá localizar películas por título. Adicionalmente, se ofrecerán filtros por género para facilitar la exploración del catálogo.

VALORACIÓN

Desde la ficha de la película, el usuario podrá puntuarla con una valoración personal del 1 al 5 (estrellas). La valoración se guarda en la base de datos mediante una llamada PATCH y se refleja al instante en la tarjeta y en la ficha.

[3.2] Gestión y mantenimiento de Datos (CRUD)

Gestión de Películas (CRUD)

- Insertar nuevas películas mediante formulario con los campos: título, sinopsis, duración, fecha de estreno, póster (URL) y género seleccionable mediante desplegable.
- Editar los datos de cualquier película existente.
- Eliminar películas del catálogo, con mensaje de confirmación previo.

[3.3] Watchlist (Pendientes de ver)

Las películas marcadas como no vistas se agrupan en la Watchlist. El usuario puede buscar entre sus pendientes y, cuando ve una, pulsar «Ya la he visto» para marcarla como vista (PATCH) y la tarjeta desaparecerá de la lista con una animación.

[3.4] Favoritas

El usuario puede marcar o quitar cualquier película como favorita desde el catálogo o la ficha. La sección Favoritas reúne todas ellas y permite quitarlas con un clic.

[3.5] Reseña personal

En la ficha de cada película el usuario puede escribir y editar una reseña personal, que se guarda de forma individual asociada a esa película.

[3.6] Estadísticas

Un panel resume la actividad del usuario en cinco indicadores: total de películas registradas, tiempo total visto, género con más películas registradas, pendientes en la watchlist y reseñas escritas.

[3.7] Paginación

El catálogo se sirve paginado (4 películas por página) mediante Spring Data Pageable, evitando cargar todo el catálogo de una sola vez.

[4] API REST y Modelo de Datos

La aplicación expone una API REST bajo /api que intercambia JSON. A continuación, se resume el modelo de datos y los endpoints disponibles, la documentación interactiva completa está en Swagger UI (/swagger-ui/index.html).

[4.1] Modelo de datos

Entidad Película

Campo	Tipo	Restricción / Notas
id	Long	Clave primaria autogenerada
titulo	String	Obligatorio (@NotBlank)
duracion	Integer	Mínimo 1 minuto (@Min)
anio	Integer	Año de estreno, mínimo 1888 (@Min)
sinopsis	String	Texto largo
actores	String	Reperto principal
urlPortada	String	URL del póster
urlTrailer	String	URL del tráiler (YouTube)
valoracion	Integer	De 1 a 5 (@Min/@Max)
favorita	Boolean	Marca de favorita
vista	Boolean	Vista (true) o pendiente (false)
resenaPersonal	String	Reseña del usuario (texto largo)
genero	Genero	Relación @ManyToOne

Entidad Género

Campo	Tipo	Restricción / Notas
id	Long	Clave primaria autogenerada
nombre	String	Obligatorio y único

[4.2] Endpoints de Películas (/api/peliculas)

Método	Ruta	Descripción
GET	/api/peliculas?buscar=&generoId=&page=0&vista=true	Listado paginado (4/pág) con filtros
GET	/api/peliculas/favoritas	Lista de favoritas
GET	/api/peliculas/{id}	Ficha de una película (404 si no existe)
POST	/api/peliculas	Crear (valida con @Valid – 201)
PUT	/api/peliculas/{id}	Reemplazar todos los datos
DELETE	/api/peliculas/{id}	Borrar (204 / 404)
PATCH	/api/peliculas/{id}/valoracion?valoracion=5	Actualizar la valoración
PATCH	/api/peliculas/{id}/vista	Alternar vista / pendiente
POST	/api/peliculas/{id}/favorita	Alternar favorita
PATCH	/api/peliculas/{id}/reseña	Guardar / editar la reseña personal

[4.3] Endpoints de Géneros (/api/generos)

Método	Ruta	Descripción
GET	/api/generos	Lista de géneros
GET	/api/generos/{id}	Género por id
POST / PUT / DELETE	/api/generos/{id}	Alta, edición y borrado de géneros

[4.4] Endpoint de Estadísticas (/api/estadisticas)

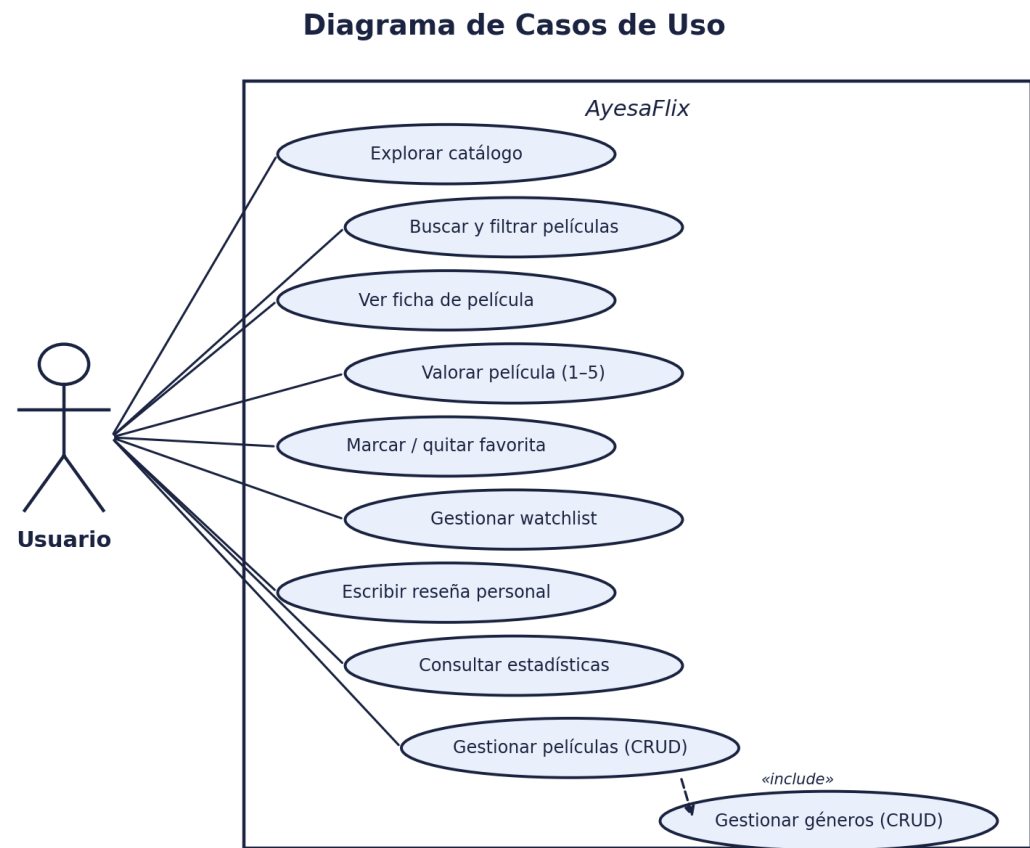
Método	Ruta	Descripción
GET	/api/estadisticas	totalRegistradas, watchlistPendientes, tiempoTotalMinutos, totalResenas, generoFavorito

[5] Diagramas UML

A continuación, se presentan los diagramas UML diseñados para documentar la arquitectura, el comportamiento y el modelo de datos del proyecto AyesaFlix. Se ha utilizado la sintaxis de PlantUML.

[5.1] Diagrama de Caso de Uso

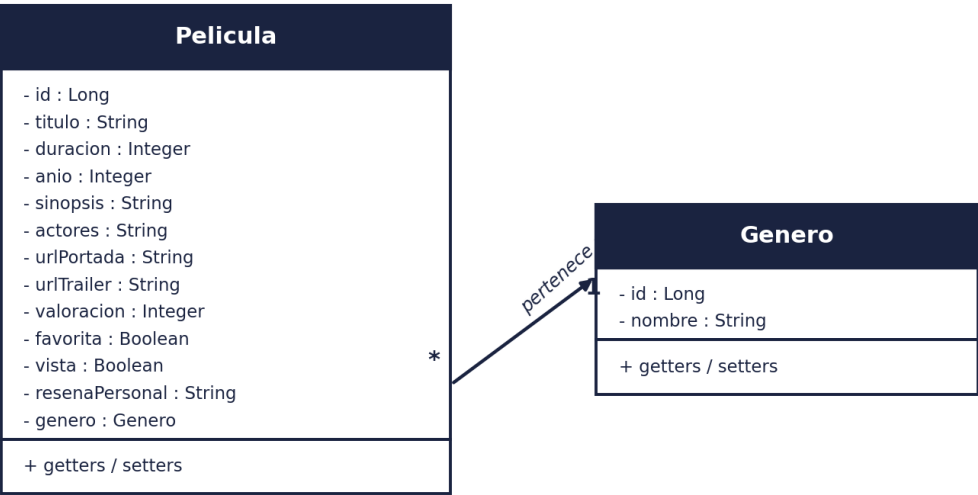
Muestra las interacciones del usuario final con las distintas funcionalidades del sistema.



[5.2] Diagrama de Clases

Representa las entidades del modelo de base de datos y su relación (1:N).

Diagrama de Clases (modelo de datos)

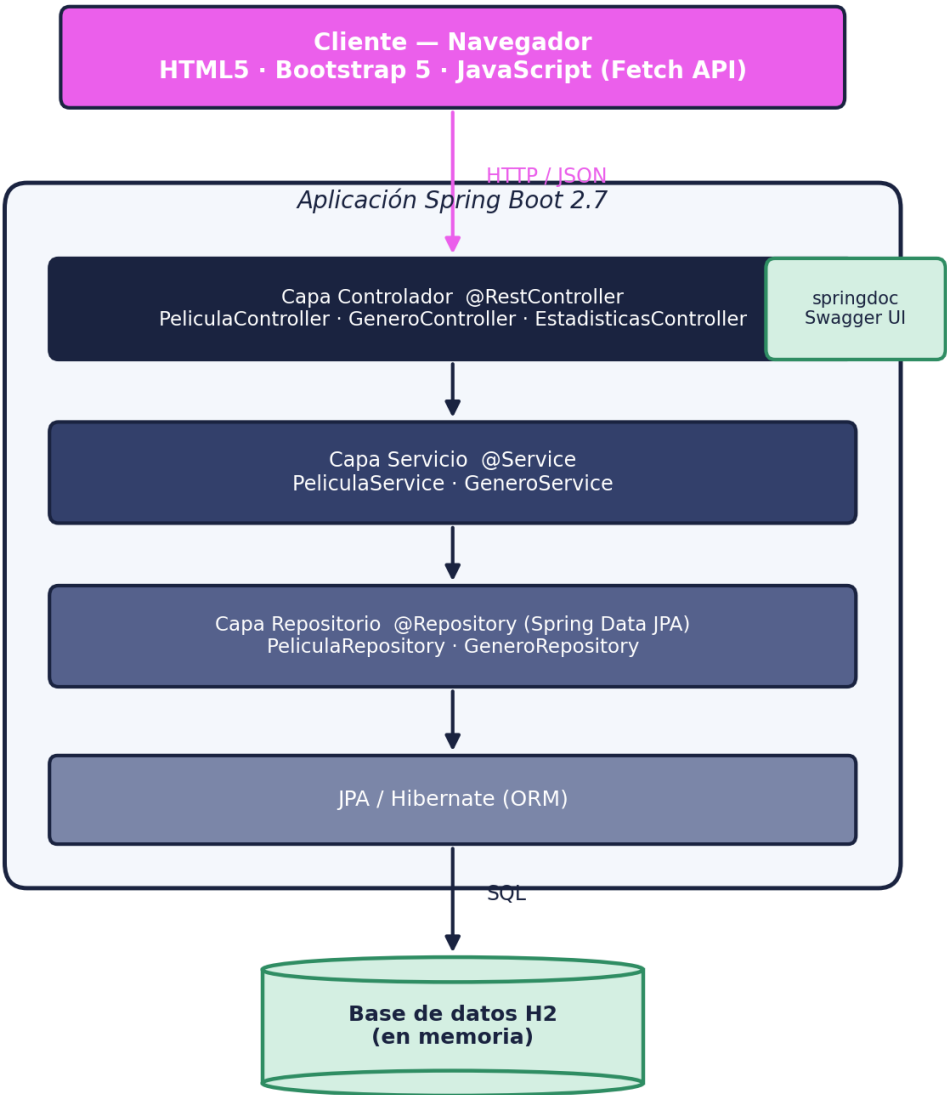


Relación @ManyToOne: muchas películas pertenecen a un género (1 : N).

[5.3] Diagrama de Arquitectura

Ilustra la arquitectura por capas de Spring Boot: el navegador consume la API REST y las peticiones recorren las capas Controlador → Servicio → Repositorio → Entidad hasta la base de datos H2.

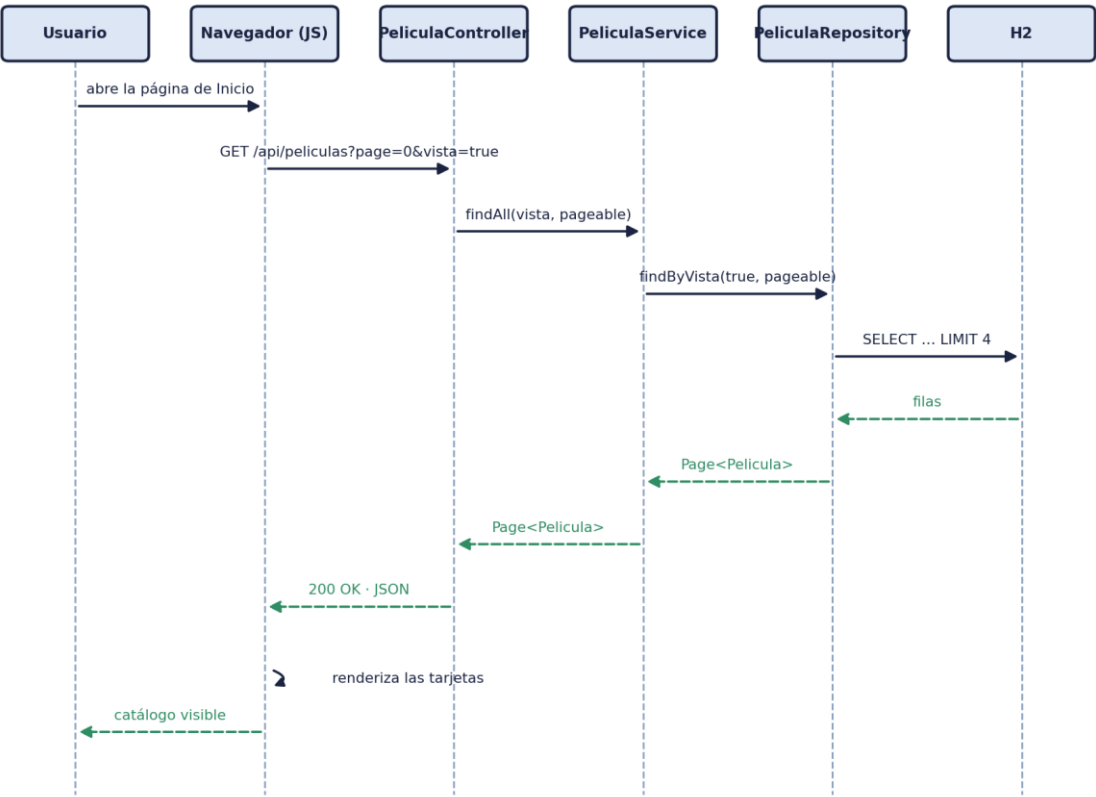
Diagrama de Arquitectura (componentes por capas)



[5.4] Diagrama de Secuencia: Carga de la Cartelera

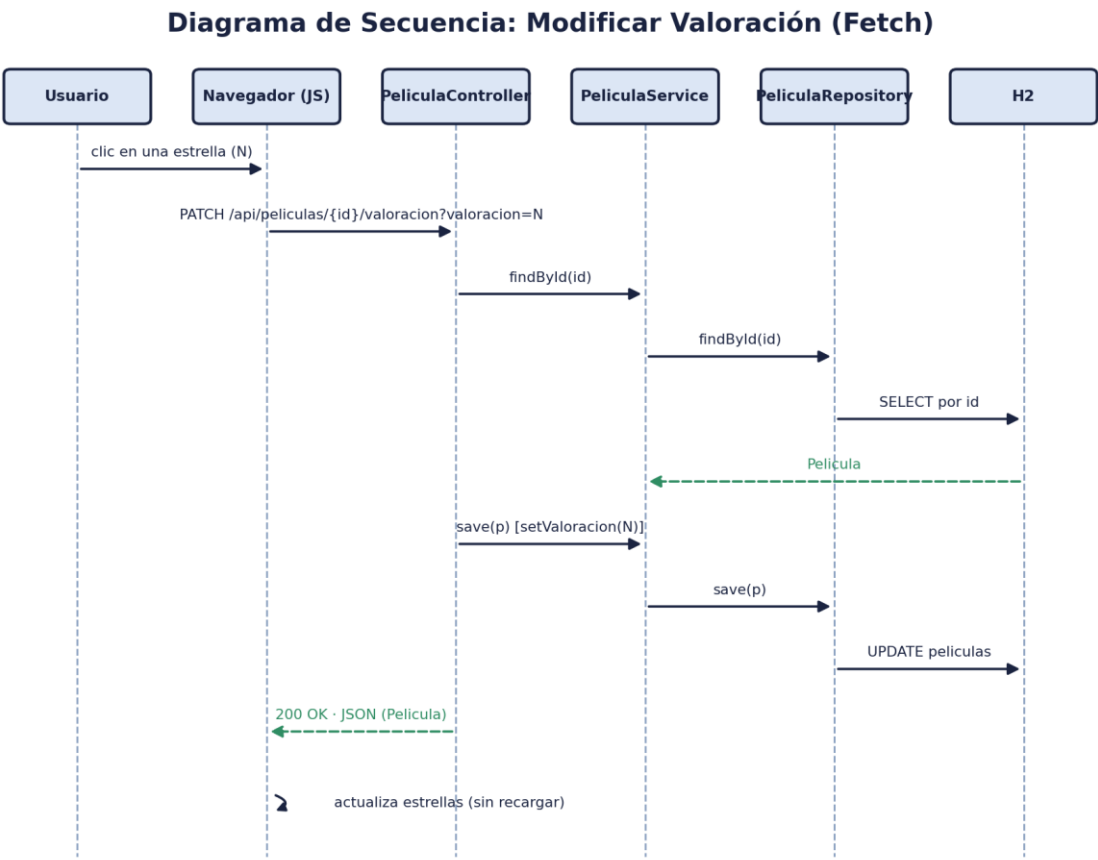
Muestra el flujo paso a paso desde que el usuario abre la página principal: el navegador solicita los datos mediante fetch (GET /api/peliculas) y recibe un JSON que se renderiza dinámicamente como tarjetas, sin recargar la página.

Diagrama de Secuencia: Carga de la Cartelera



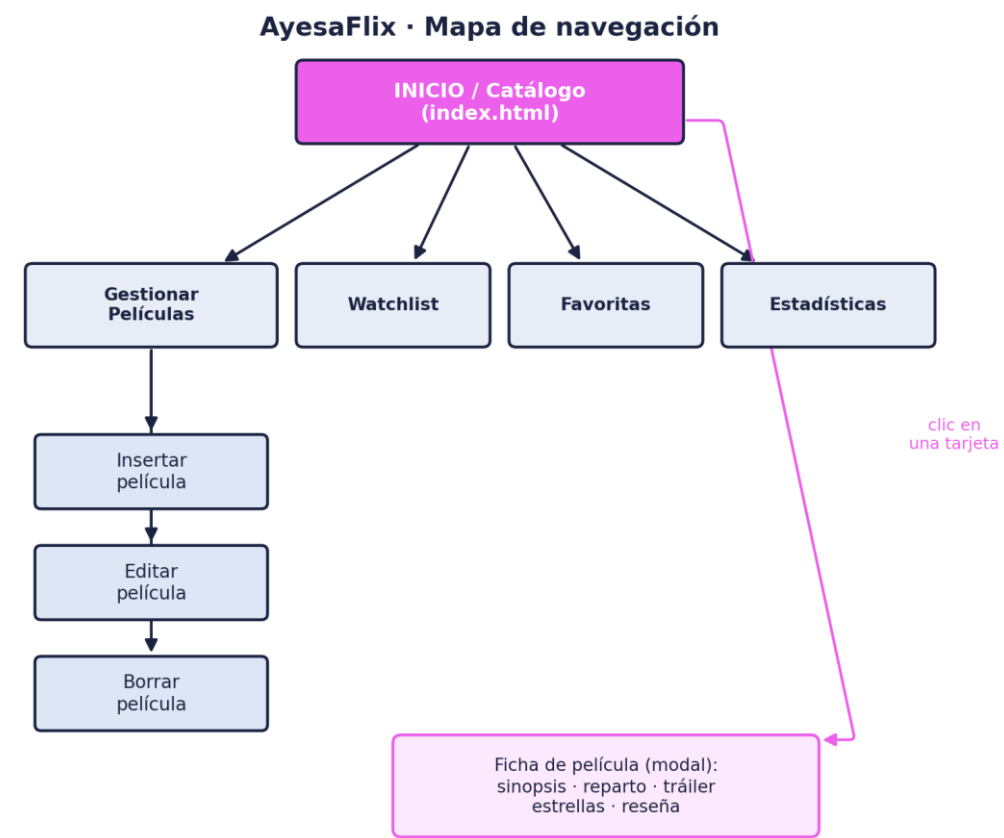
[5.5] Diagrama de Secuencia: Modificar Valoración (Fetch)

Explica cómo la interacción asíncrona de JavaScript interactúa con el backend sin recargar la página.



[5.6] Diagramas de Flujo

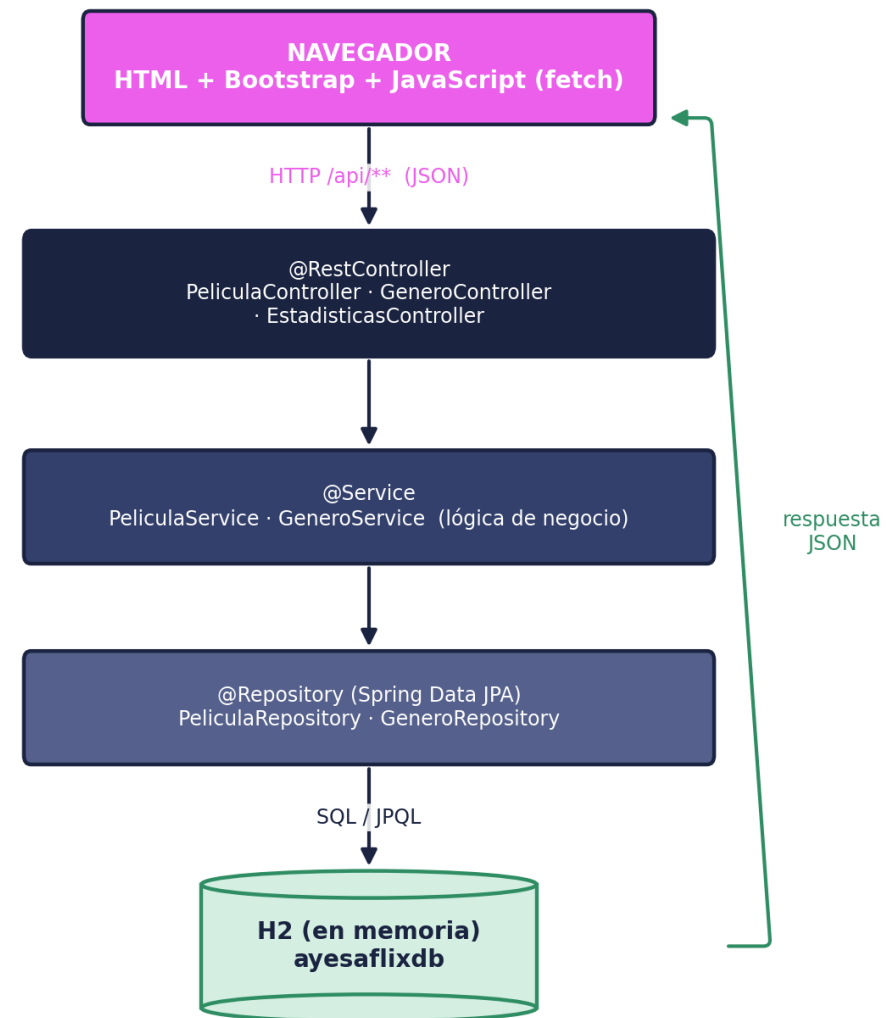
Los siguientes diagramas de flujo describen los principales recorridos del usuario y la comunicación entre el frontend y la API REST.



El menú superior (layout.html) está presente en todas las pantallas.

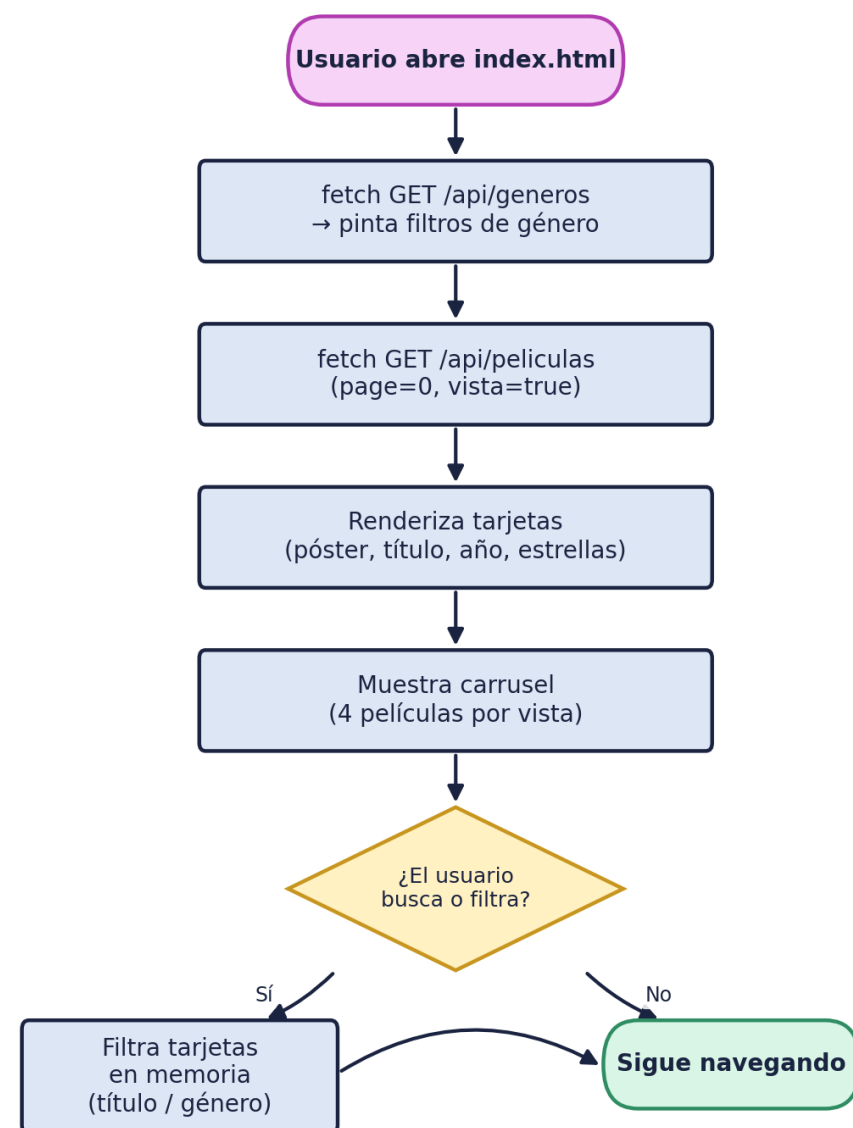
Mapa de navegación entre pantallas.

Arquitectura por capas · flujo de una petición REST



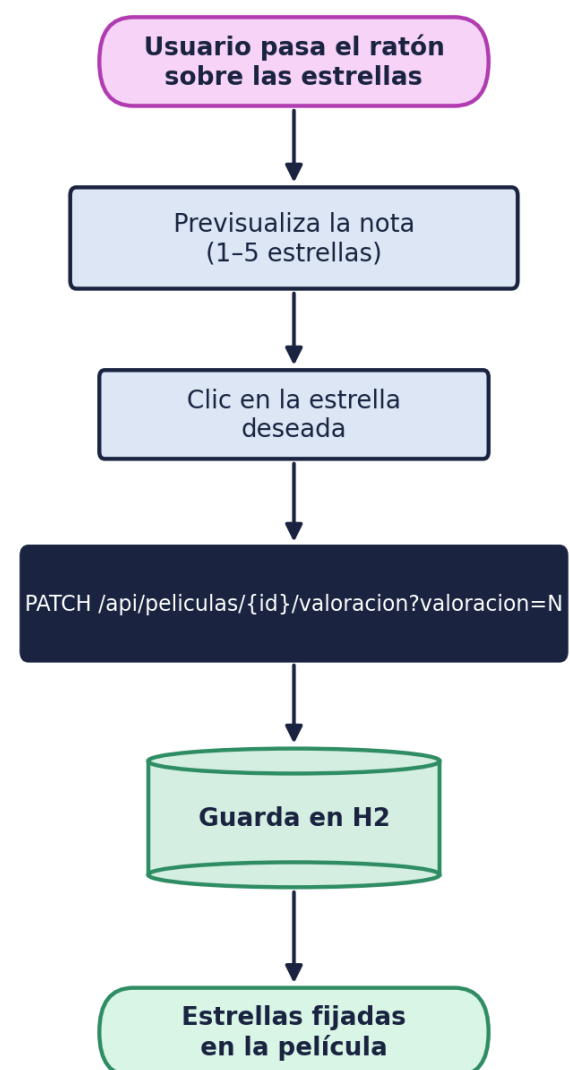
Arquitectura por capas y flujo de una petición REST.

Flujo: carga del catálogo (Inicio)



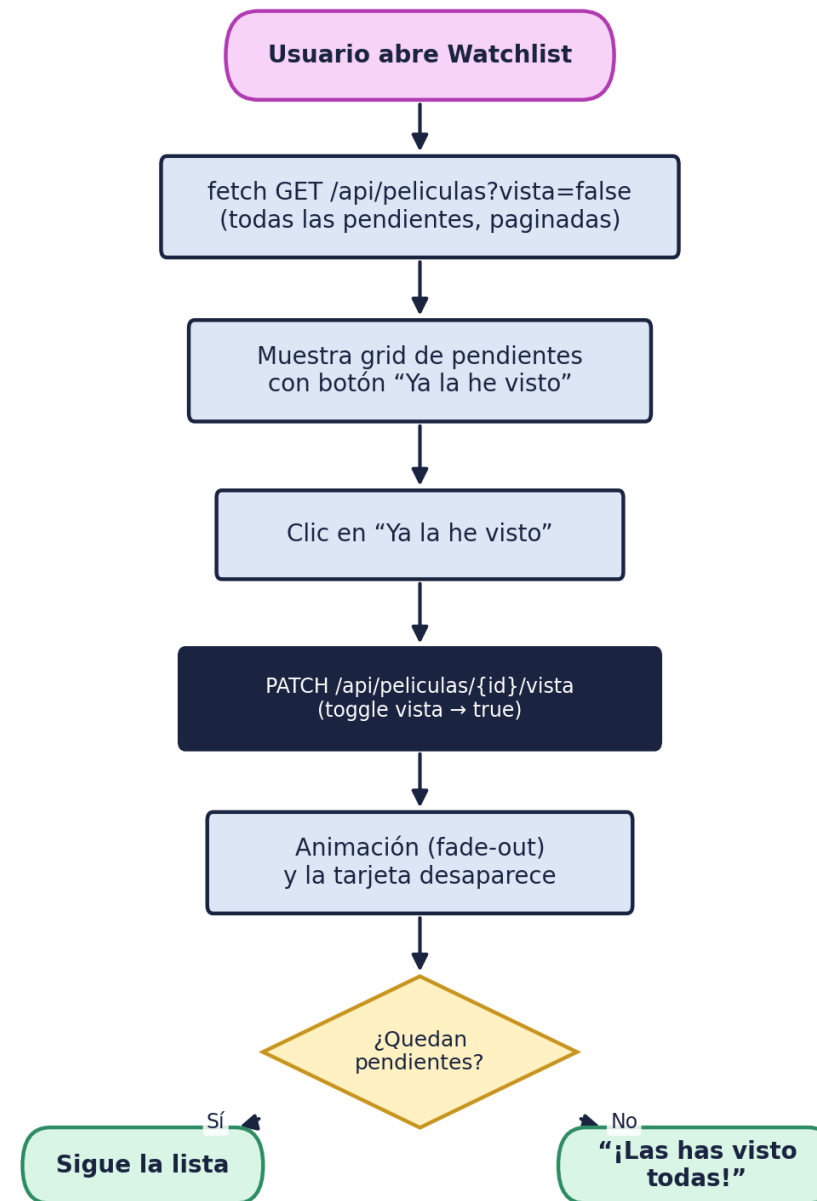
Carga del catálogo en la página de inicio.

Flujo: valorar película (estrellas)



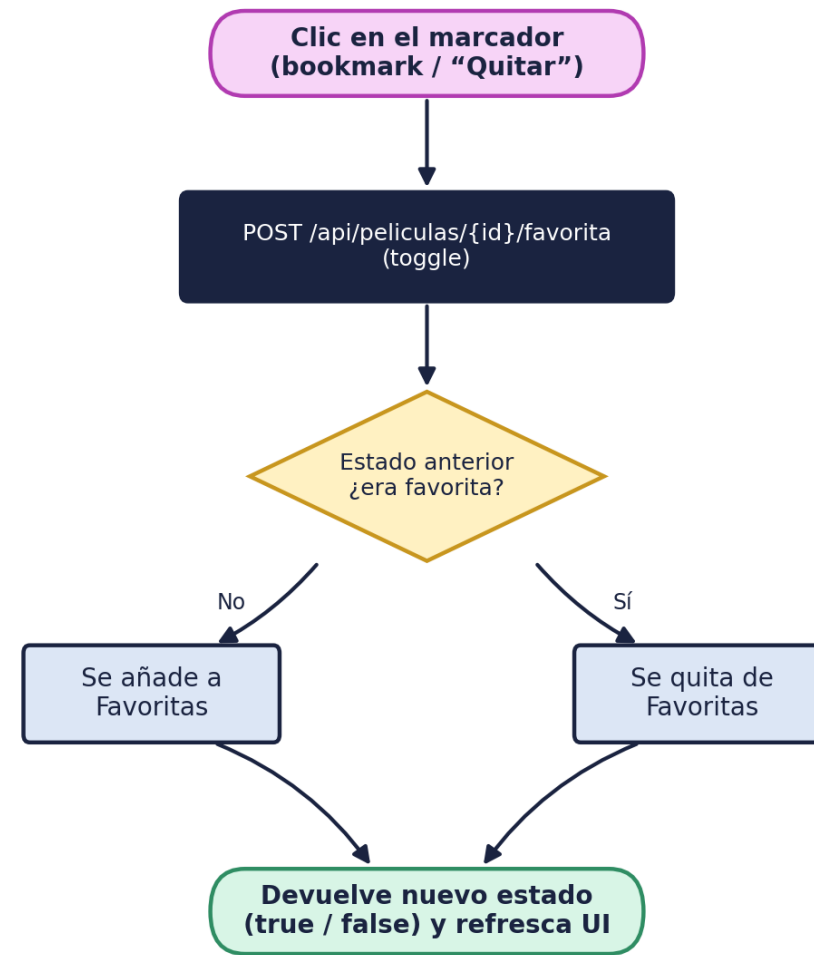
Valorar una película con estrellas.

Flujo: Watchlist · marcar como vista



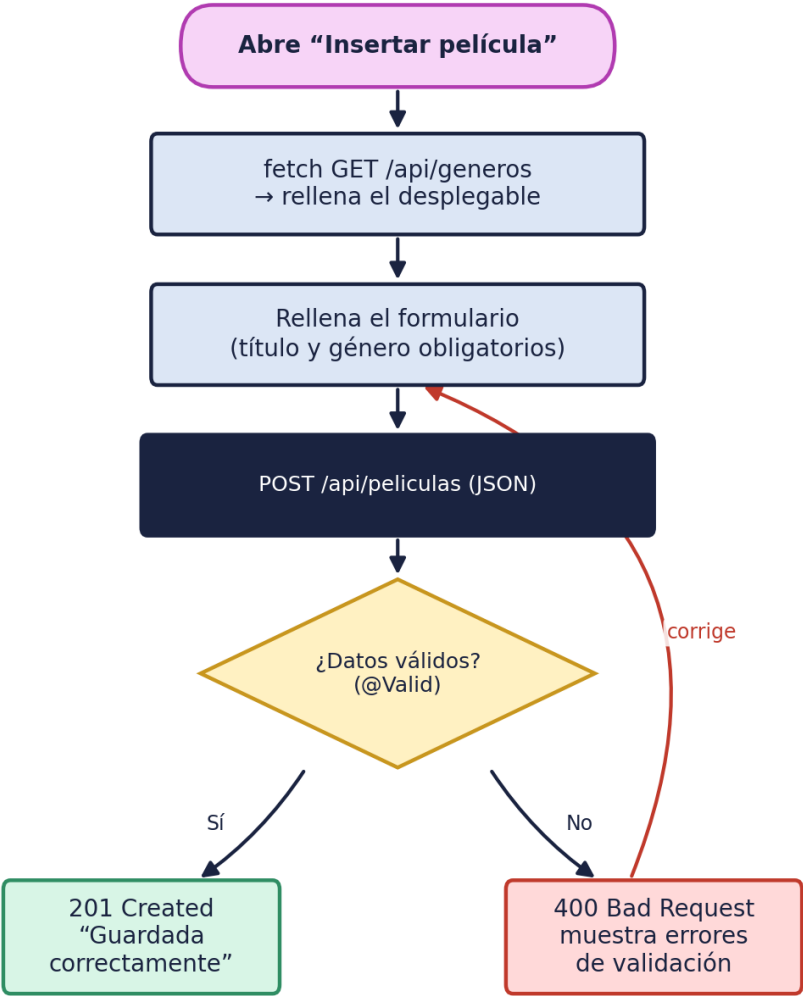
Watchlist: marcar una película como vista.

Flujo: marcar / quitar favorita

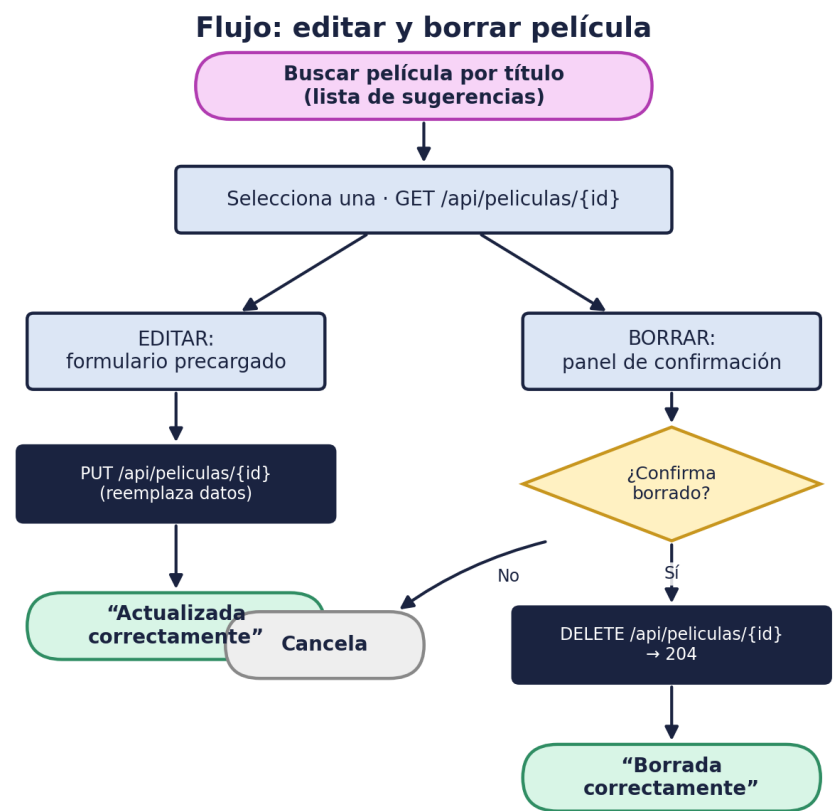


Marcar o quitar una película de favoritas.

Flujo: crear película (Insertar)

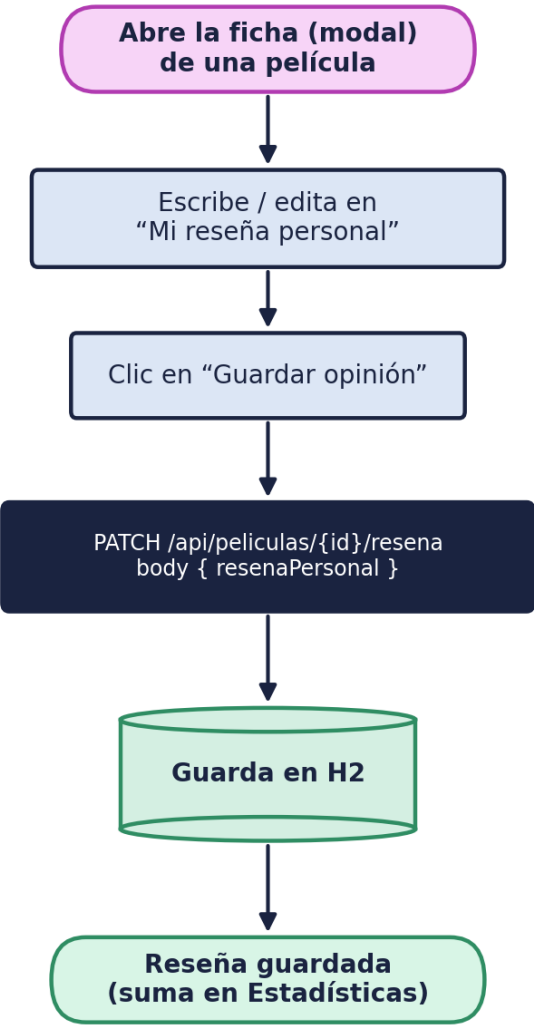


Crear una película (con validación).



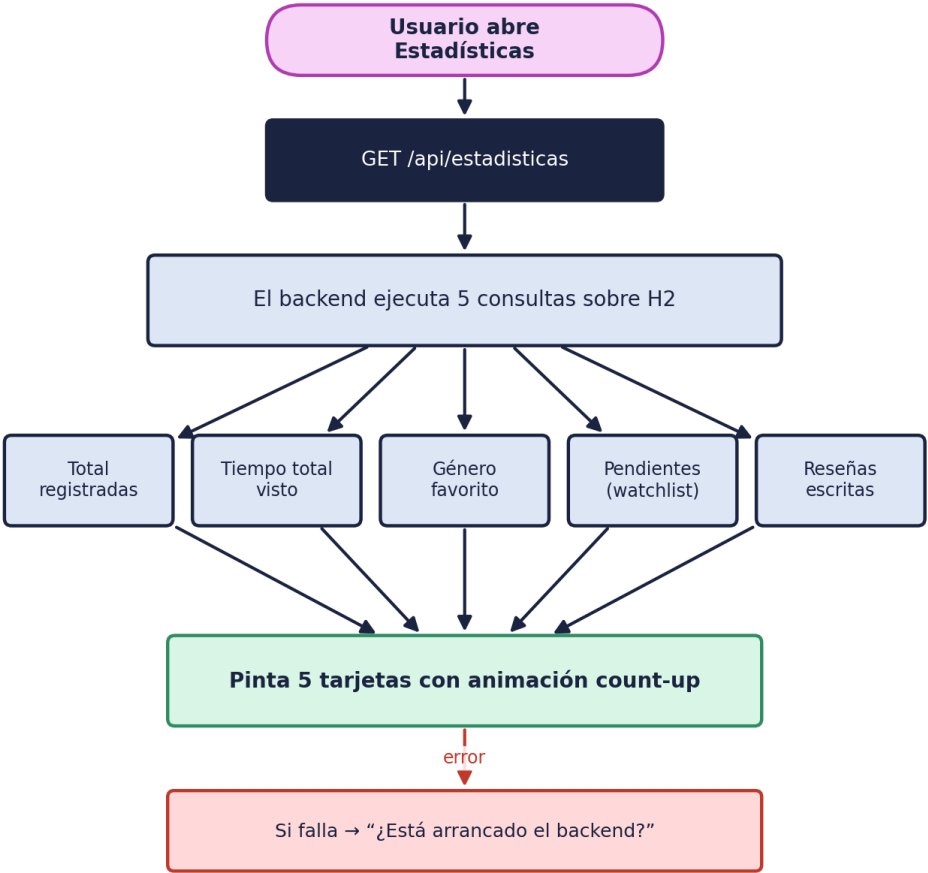
Editar y borrar una película.

Flujo: escribir reseña personal



Escribir una reseña personal.

Flujo: panel de estadísticas



Panel de estadísticas.

[6] Guía despliegue

Requisitos previos: JDK 8 o superior. No es necesario instalar Maven, ya que el proyecto incluye el wrapper (mvnw).

1. Clonar el repositorio:

git clone <https://gitlab.com/miguelpolvillo.dev/ayesaflux.git>

2. Arrancar la aplicación desde la carpeta del proyecto

Windows: mvnw.cmd spring-boot:run

Linux / macOS: ./mvnw spring-boot:run

3. Acceder a la aplicación

Aplicación web: <http://localhost:8080/>

Swagger UI (API): <http://localhost:8080/swagger-ui/index.html>

Consola H2: <http://localhost:8080/h2-console> (JDBC URL: jdbc:h2:mem:ayesafluxdb, usuario: sa, sin contraseña)

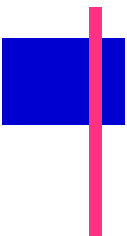
La base de datos es en memoria: en cada arranque se recrea y se rellena con data.sql (7 géneros y 15 películas de ejemplo).

4. Ejecutar los tests

mvnw.cmd test

5. Probar la API con Postman

Importar documentacion/AyesaFlix_Postman.json.



[7] Equipo

[7.1] Backend

Iván Zamora Jiménez
Miguel Polvillo Acevedo
Víctor Coquelet Salguero
Alejandro Aguilar Santaella

[7.2] Base de Datos

Daniel Vázquez Chinchilla

[7.3] Front-End

Miguel Castillero Gómez
Abderrazak Oumouss Ait Moussa
Carlos León Saavedra